

월급 생존 OS

아래는 “월급 생존 OS” 로컬 개발 시제품 v0.1 기획·구조·디자인 가이드입니다.

방향은 실제 금융 연동 없이, 고객이 체감하는 UI/UX와 시각화 기능을 먼저 검증하는 웹 시제품입니다.

1. 제품 정의

서비스 한 줄 설명

월급 생존 OS는 월급날부터 다음 월급날까지 “내 돈이 며칠 더 버틸 수 있는지”를 보여주는 사회초년생용 월급 생존 대시보드입니다.

기존 가계부가 “무엇을 썼는지 기록하는 앱”이라면, 월급 생존 OS는 다음 질문에 답해야 합니다.

“이번 달 나 진짜 괜찮아?”

“카드값, 월세, 구독료 빠지고 나면 하루에 얼마 써도 돼?”

“지금 이거 사도 다음 월급날까지 버틸 수 있어?”

2. 왜 이 서비스가 필요한가

2030 사회초년생은 1인 가구, 월세, 카드값, 구독료, 식비, 교통비, 청약·적금 같은 반복 지출을 동시에 관리해야 합니다. 통계청의 2024년 1인가구 통계에서 2023년 1인가구는 전체 가구의 35.5%였고, 연령대별로 29세 이하 18.6%, 30대 17.3%가 큰 비중을 차지했습니다. 즉 2030의 “혼자 월급 관리”는 꽤 큰 시장입니다. ([국가데이터처](#))

구독 서비스도 좋은 진입점입니다. 서울시 실태조사 보도에 따르면 구독 서비스 이용자 중 95.9%가 하나 이상 구독 중이고, 월평균 지출액은 약 4만 원대였습니다. 특히 20대와 30대의 구독 지출이 높게 나타났고, 무료체험 후 유료 전환이나 해지 불편 같은 문제가 보고됐습니다. ([뉴시스](#))

금융 리스크도 있습니다. 2024년 청년금융 실태조사를 인용한 보도에서는 청년 연체자, 장기연체, 다중채무, 리볼빙 사용 문제가 언급됐습니다. 이 서비스는 투자 추천보다 **카드값·고정비·리볼빙 위험을 조기에 시각화하는 생활 방어형 서비스**로 설계하는 것이 안전합니다. ([아시아경제](#))

커뮤니티 검색 결과에서도 “가계부 앱은 광고가 많고 뭘 써야 할지 모르겠다”, “보기 편하고 정리 쉬운 앱을 찾고 싶다”, “월급 계획을 누군가 봐줬으면 좋겠다”는 니즈가 반복됩니다. 블라인드 게시물 검색 결과에는 사회초년생 월급 계획 피드백, 가계부 앱 추천, 자동 연동·공유 관리의 번거로움 같은 사례가 보입니다. (Blind)

3. 핵심 고객

1차 고객

24~32세 사회초년생

특징은 다음과 같습니다.

항목	내용
소득	월 실수령 200만~350만 원대
고민	월급은 들어오는데 금방 사라짐
반복 지출	월세, 관리비, 통신비, 교통비, 식비, 구독료, 카드값
행동 패턴	토스·카카오페이·카드 앱은 보지만 전체 생존 가능성은 모름
결제 허들	월 3,900원은 “내가 한 달에 3,900원 이상 아끼는 게 보이면” 가능

2차 고객

30대 1인 가구 직장인 / 결혼 전 커플 / 신혼 초 공동지출 관리층

초기 시제품에서는 1인용으로 만들고, 나중에 “커플·신혼 공동 생존 OS”로 확장하는 것이 좋습니다.

4. 제품 핵심 콘셉트

제품 이름

월급 생존 OS

핵심 메시지

“월급날까지 남은 생존일, 하루 사용 가능 금액, 빠질 돈을 한눈에.”

유료 전환 명분

월 3,900원을 받으려면 사용자가 매달 이 감정을 느껴야 합니다.

“이번 달에 이 서비스 덕분에 3,900원보다 훨씬 더 아꼈다.”

그래서 홈 화면에 반드시 이 지표를 넣어야 합니다.

이번 달 발견한 새는 돈: 27,400원

이번 달 3,900원 값어치 달성률: 702%

5. MVP 기능 범위

로컬 시제품은 처음부터 은행·카드 API를 붙이지 않습니다. 실제 금융정보 연동은 금융 마이데이터, 표준 API, 개인정보·신용정보 규정 검토가 필요한 영역입니다. 금융위원회 자료에서도 마이데이터 사업자는 표준 API를 통해 개인신용정보를 수집·활용하는 구조로 설명됩니다. 따라서 시제품은 **목업 데이터, 수동 입력, CSV 업로드** 중심으로 가야 합니다. (KDI 경제정보센터)

v0.1 필수 기능 6개

기능	설명	UI 핵심
1. 월급 세팅	급여일, 실수령액, 고정비, 저축 목표 입력	3단계 온보딩
2. 생존 대시보드	다음 월급날까지 남은 돈과 위험도 표시	생존 게이지
3. 고정비 타임라인	월세·관리비·카드값·구독료 빠지는 날짜 표시	돈 빠지는 달력
4. 하루 사용 가능 금액	오늘부터 월급날까지 하루 얼마 써도 되는지 계산	Daily Allowance 카드
5. 구독·자동결제 레이더	OTT, 멤버십, 음악, 앱 구독료 관리	구독 뱃지, 해지 체크
6. 지금 사도 돼?	금액 입력 시 이번 달 생존 영향 시뮬레이션	구매 판단 모달

6. 사용자 경험 흐름

flowchart TD

A[첫 접속] --> B[3분 월급 세팅]

B --> C[급여일·실수령액 입력]

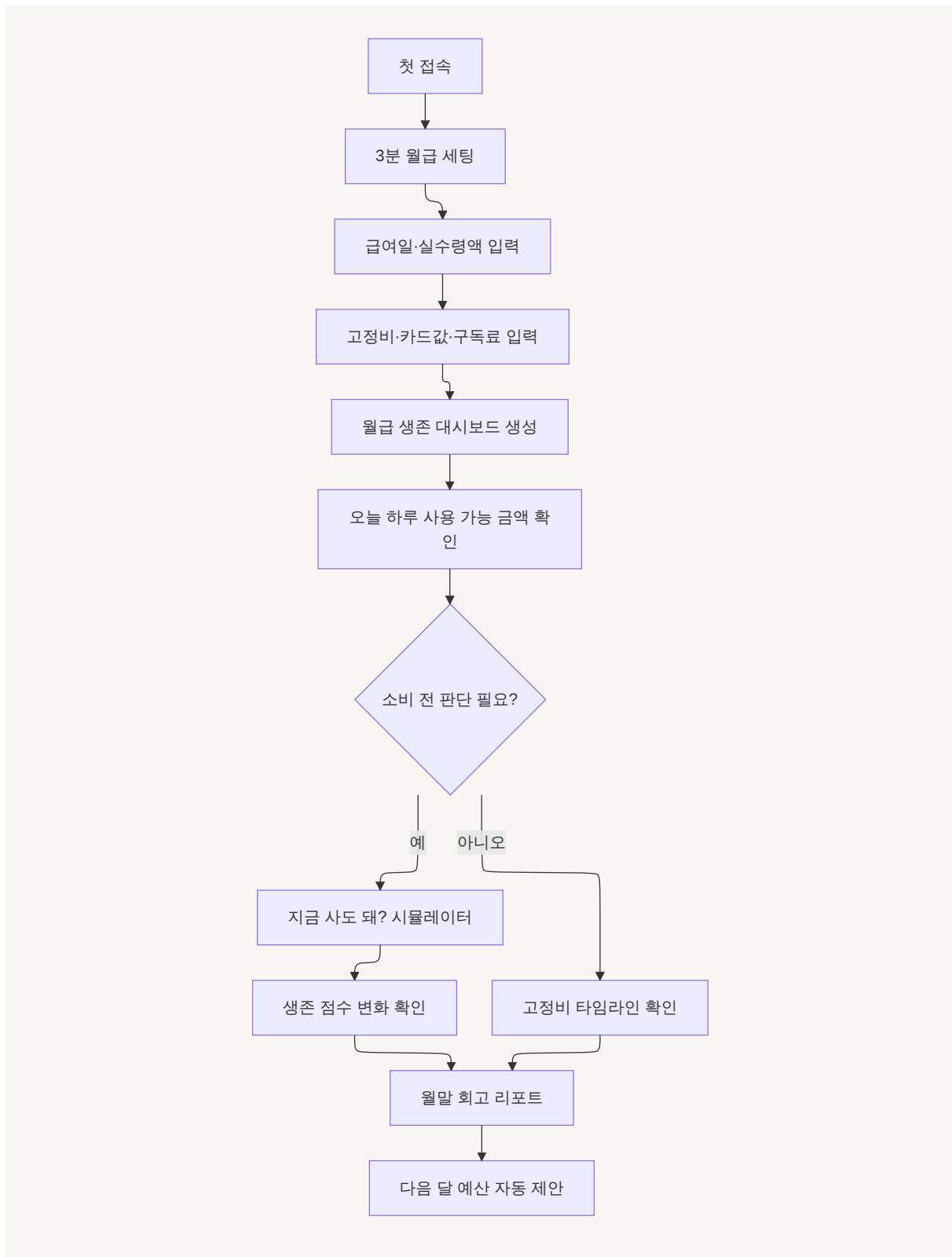
C --> D[고정비·카드값·구독료 입력]

D --> E[월급 생존 대시보드 생성]

E --> F[오늘 하루 사용 가능 금액 확인]

F --> G{소비 전 판단 필요?}

G --> |예| H[지금 사도 돼? 시뮬레이터]
G --> |아니오| I[고정비 타임라인 확인]
H --> J[생존 점수 변화 확인]
I --> K[월말 회고 리포트]
J --> K
K --> L[다음 달 예산 자동 제안]



7. 정보 구조

graph TD

Root[월급 생존 OS] --> Onboarding[온보딩]

Root --> Dashboard[홈 대시보드]

Root --> Timeline[돈 빠지는 달력]

Root --> Budget[생활비 버킷]

Root --> Subscription[구독 레이더]

Root --> Decision[지금 사도 돼?]

Root --> Review[월말 회고]

Root --> Settings[설정]

Onboarding --> Salary[월급 정보]

Onboarding --> Fixed[고정비 입력]

Onboarding --> Goals[저축·비상금 목표]

Dashboard --> SurvivalScore[생존 점수]

Dashboard --> DailyAllowance[하루 사용 가능 금액]

Dashboard --> UpcomingBills[다가오는 결제]

Dashboard --> LeakFinder[새는 돈 발견]

Timeline --> Calendar[캘린더]

Timeline --> DangerWeek[위험 주간]

Budget --> Food[식비]

Budget --> Cafe[카페]

Budget --> Transport[교통]

Budget --> Shopping[쇼핑]

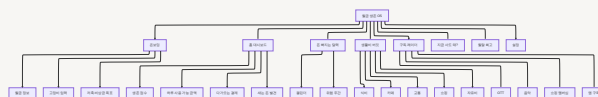
Budget --> Flex[자유비]

Subscription --> OTT[OTT]

Subscription --> Music[음악]

Subscription --> Membership[쇼핑 멤버십]

Subscription --> App[앱 구독]



8. 로컬 시제품 시스템 구조도

권장 기술 스택

영역	추천
프론트엔드	Next.js 또는 Vite + React
언어	TypeScript
스타일	Tailwind CSS
UI 컴포넌트	shadcn/ui 또는 Radix UI
차트	Recharts
상태관리	Zustand
로컬 DB	SQLite
ORM	Prisma
데이터 입력	Mock seed data + CSV import
인증	로컬 프로필만 사용
결제	실제 결제 없음, 3,900원 paywall 목업

구조도

flowchart LR

User[사용자] --> Browser[브라우저 / PWA]

Browser --> UI[React UI Layer]

UI --> State[Zustand State]

UI --> Charts[Recharts 시각화]

UI --> API[Local API Routes]

API --> Rules[월급 생존 계산 엔진]

API --> Importer[CSV / Mock Data Importer]

API --> DB[(SQLite Local DB)]

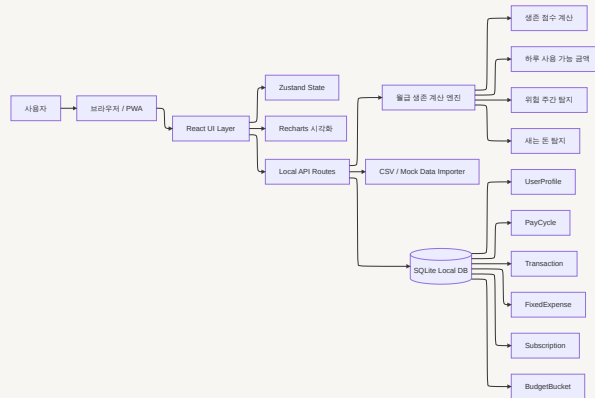
Rules --> Score[생존 점수 계산]

Rules --> Allowance[하루 사용 가능 금액]

Rules --> Risk[위험 주간 탐지]

Rules --> Leak[새는 돈 탐지]

DB --> Profile[UserProfile]
 DB --> PayCycle[PayCycle]
 DB --> Transaction[Transaction]
 DB --> FixedExpense[FixedExpense]
 DB --> Subscription[Subscription]
 DB --> BudgetBucket[BudgetBucket]



9. 데이터 모델

ERD

erDiagram

```
UserProfile ||--o{ PayCycle : has
UserProfile ||--o{ FixedExpense : owns
UserProfile ||--o{ Subscription : owns
UserProfile ||--o{ BudgetBucket : owns
UserProfile ||--o{ Transaction : records
PayCycle ||--o{ Insight : generates
```

```
UserProfile {
    string id
    string nickname
    int salaryDay
    int monthlyIncome
    int emergencyFloor
    string riskPreference
```



```
        datetime createdAt
    }

    PayCycle {
        string id
        string userId
        date startDate
        date endDate
        int income
        int remainingCash
        int survivalScore
        int dailyAllowance
    }

    FixedExpense {
        string id
        string userId
        string name
        int amount
        int dueDay
        string category
        boolean isEssential
    }

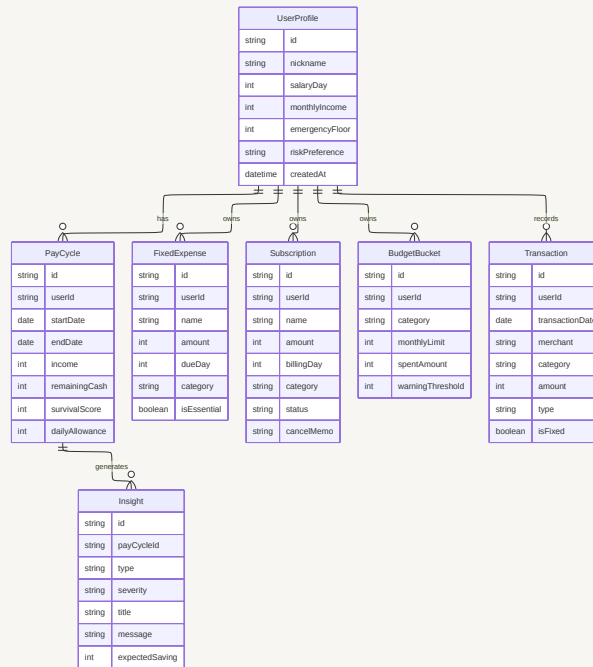
    Subscription {
        string id
        string userId
        string name
        int amount
        int billingDay
        string category
        string status
        string cancelMemo
    }

    BudgetBucket {
        string id
        string userId
    }
```

```
    string category
    int monthlyLimit
    int spentAmount
    int warningThreshold
}
```

```
Transaction {
    string id
    string userId
    date transactionDate
    string merchant
    string category
    int amount
    string type
    boolean isFixed
}
```

```
Insight {
    string id
    string payCycleId
    string type
    string severity
    string title
    string message
    int expectedSaving
}
```



10. 핵심 계산 로직

10-1. 생존 가능 금액

남은_생존금액 =
 현재잔액
 - 다음월급전_빠질_고정비
 - 다음월급전_카드값
 - 비상금_보존선

10-2. 하루 사용 가능 금액

하루_사용가능금액 =
 남은_생존금액 / 다음_월급일까지_남은_일수

10-3. 생존 점수

100점에서 시작해서 위험 요소를 차감합니다.

위험 요소	차감
고정비 비율이 높음	-5 ~ -20
다음 7일 내 큰 결제 예정	-5 ~ -15
생활비 버킷 초과	-5 ~ -20
구독료가 많음	-3 ~ -10
카드값이 남은 현금 대비 큼	-10 ~ -25
비상금 보존선 아래	-10 ~ -30

10-4. 위험 주간 탐지

위험주간 =

특정 7일 동안 빠질 돈 > 현재 잔액의 30%

또는

특정 7일 이후 하루 사용 가능 금액 < 사용자 설정 최소 생활비

11. 화면 설계

11-1. 온보딩

목표는 3분 안에 첫 생존 대시보드 생성입니다.

Step 1. 월급 정보

입력 항목:

항목	예시
월급일	매월 25일
실수령액	2,650,000원
현재 잔액	1,840,000원
다음 월급일	자동 계산

UX 문구:

“정확하지 않아도 괜찮아요. 먼저 대략 입력하고 나중에 수정할 수 있어요.”

Step 2. 고정비

입력 항목:

항목	예시
월세	650,000원
관리비	120,000원
통신비	55,000원
교통비	80,000원
보험료	70,000원
카드값	430,000원

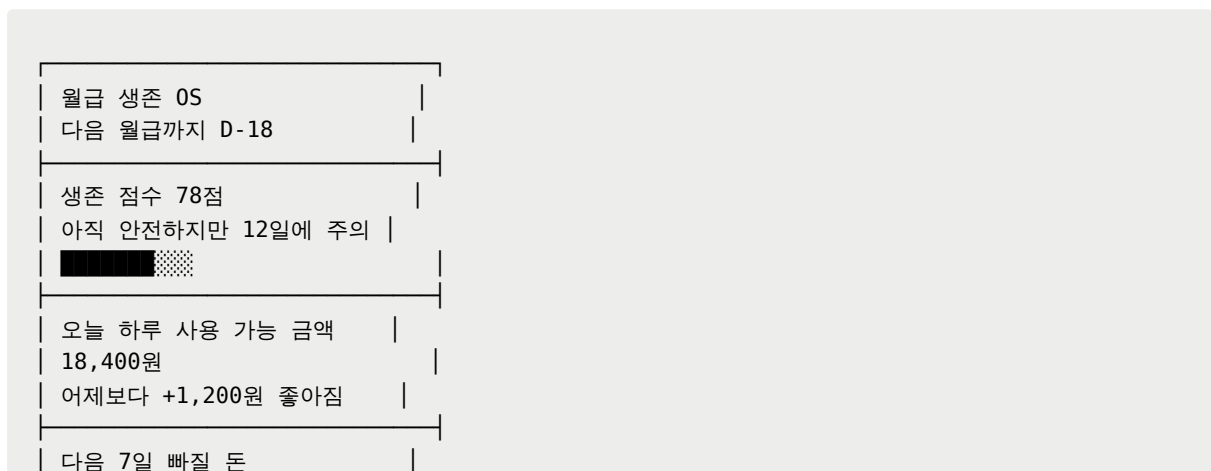
Step 3. 구독료

빠른 선택 버튼:

- 넷플릭스
- 유튜브 프리미엄
- 쿠팡 와우
- 멜론
- 티빙
- 디즈니+
- 네이버플러스
- 기타 직접 입력

11-2. 홈 대시보드

핵심 화면입니다. 사용자가 매일 들어와서 10초 안에 상태를 이해해야 합니다.



① 통신비 55,000원 D-2	
② 넷플릭스 17,000원 D-4	
③ 카드값 430,000원 D-7	
이번 달 발견한 새는 돈	
27,400원	
3,900원 값어치 달성률 702%	

홈 화면 구성 요소

컴포넌트	역할
SurvivalGauge	생존 점수 시각화
DailyAllowanceCard	하루 사용 가능 금액
UpcomingBillsCard	다음 7일 빠질 돈
LeakFinderCard	새는 돈 발견
QuickDecisionButton	"지금 사도 돼?" 진입
BudgetBucketPreview	식비·카페·쇼핑 잔여 한도

11-3. 돈 빠지는 달력

고객이 가장 직관적으로 이해할 수 있는 화면입니다.

화면 목표

"언제 돈이 빠져나가서 위험해지는지"를 날짜로 보여줍니다.

4월						
월	화	수	목	금	토	일
1	2	3	4	5	6	7
		통신		OTT		
8	9	10	11	12	13	14
				카드		

날짜 색상 규칙

상태	의미
초록	안전

상태	의미
노랑	결제 예정 있음
주황	하루 사용 가능 금액 하락
빨강	위험 결제일
회색	이미 지난 날짜

색만으로 판단하지 않도록 반드시 라벨을 같이 붙입니다.

예:

- 안전
- 주의
- 위험
- 카드값
- 월세

11-4. 생활비 버킷

사회초년생에게는 카테고리별 잔여 한도가 중요합니다.

이번 달 생활비 버킷

식비

한도 350,000원 / 사용 214,000원

남음 136,000원



카페

한도 80,000원 / 사용 72,000원

남음 8,000원



주의: 이번 주 카페 소비가 빠릅니다.

쇼핑

한도 150,000원 / 사용 188,000원

초과 38,000원



다음 달 한도 조정 후보

버킷 기본값

버킷	설명
식비	편의점, 배달, 장보기, 외식
카페	커피, 디저트
교통	대중교통, 택시
생활	생필품, 세탁, 약국
쇼핑	옷, 잡화, 전자기기
관계비	데이트, 경조사, 선물
자유비	분류 애매한 소비

11-5. 구독 레이더

구독 서비스는 “금액이 작아서 방치되지만 합치면 커지는 지출”로 보여줘야 합니다.

구독 레이더

이번 달 구독료 합계
44,900원

서비스	결제일	금액
쿠팡 와우	매월 3일	7,890
유튜브 프리미엄	매월 8일	14,900
넷플릭스	매월 15일	17,000
멜론	매월 22일	5,100

추천:

최근 30일 사용하지 않은 구독이 있다면 해지 후보로 표시하세요.

구독 레이더 UX

기능	설명
다음 결제 D-day	자동결제 전 알림
해지 후보	사용 빈도 낮은 구독 표시
중복 구독	OTT 2개 이상, 음악 2개 이상 감지
3,900원 회수 계산	해지 후보 금액과 서비스 가격 비교

11-6. “지금 사도 돼?” 시뮬레이터

서비스의 대표 기능으로 만들면 좋습니다.

사용 흐름

1. 사용자가 사고 싶은 금액 입력
2. 카테고리 선택
3. 생존 점수 변화 확인
4. 대안 제안

지금 사도 돼?

구매 금액
[59,000원]

카테고리
[쇼핑]

결과
생존 점수 78점 → 71점

영향
하루 사용 가능 금액
18,400원 → 15,100원

추천
오늘 사면 다음 카드값 주간이 위험해져요.
3일 뒤 카드값 결제 후 다시 확인해보는 게 안전해요.

판단 결과 3단계

단계	문구
가능	"사도 괜찮아요. 생존 점수 변화가 작아요."
주의	"살 수는 있지만 이번 주 예산이 빠듯해져요."
보류	"지금 사면 다음 월급 전 위험 구간에 들어가요."

중요한 점은 사용자를 혼내지 않는 것입니다.

"낭비", "충동구매", "돈 관리 못함" 같은 표현은 금지합니다.

12. 시각 디자인 가이드

디자인 키워드

키워드	설명
안심	사용자가 혼나는 느낌이 아니라 통제감을 느껴야 함
직관	숫자보다 상태를 먼저 보여줌
현실적	무조건 아끼라는 말보다 이번 달 생존 가능성을 보여줌
가벼움	3,900원 서비스답게 과하게 무거운 금융앱 느낌은 피함
모바일 우선	웹이지만 모바일 화면에서 가장 먼저 잘 보여야 함

컬러 시스템

용도	색상	HEX
Primary	생존 그린	#16A34A
Warning	주의 옐로우	#F59E0B
Danger	위험 레드	#EF4444
Navy Text	본문 진한색	#111827
Sub Text	보조 텍스트	#6B7280
Background	배경	#F8FAFC
Card	카드 배경	#FFFFFF
Border	구분선	#E5E7EB
Info	정보 블루	#2563EB

점수별 상태 색상

점수	상태	색상
80~100	안전	Green
60~79	주의	Yellow
40~59	위험	Orange/Red
0~39	긴급	Red

타이포그래피

추천 폰트:

- Pretendard
- SUIT

- Noto Sans KR

글자 크기

요소	크기
대시보드 핵심 숫자	32~40px
카드 제목	16~18px
본문	14~16px
보조 설명	12~13px
버튼	15~16px

숫자는 크게, 설명은 짧게 갑니다.

예:

18,400원
오늘 하루 사용 가능 금액

레이아웃 규칙

항목	규칙
기본 화면 폭	모바일 390px 기준
카드 radius	20px
카드 padding	16~20px
화면 여백	20px
카드 간격	12~16px
버튼 높이	48~56px
하단 탭	5개 이하

하단 탭 구조

탭	이름
1	홈
2	달력
3	버킷

탭	이름
4	구독
5	설정

“지금 사도 돼?”는 하단 탭이 아니라 홈의 큰 CTA 버튼으로 둡니다.

13. 컴포넌트 디자인 가이드

13-1. SurvivalGauge

역할: 이번 달 상태를 한눈에 보여줌.

표시 정보:

- 생존 점수
- 상태 라벨
- 다음 월급 D-day
- 핵심 위험 요약

생존 점수 78점

아직 안전해요

다음 월급까지 D-18

주의: 12일 카드값 결제 후 하루 한도 감소

13-2. DailyAllowanceCard

역할: 사용자가 오늘 얼마까지 써도 되는지 알려줌.

오늘 하루 사용 가능 금액

18,400원

어제보다 +1,200원 좋아졌어요

상태별 문구:

상태	문구
좋아짐	“어제보다 여유가 생겼어요.”

상태	문구
유지	"계획대로 잘 가고 있어요."
나빠짐	"이번 주는 조금 조심하면 좋아요."

13-3. UpcomingBillsCard

역할: 다음 7일 지출 경고.

다음 7일 빠질 돈
502,000원

D-2 통신비 55,000원
D-4 넷플릭스 17,000원
D-7 카드값 430,000원

13-4. LeakFinderCard

역할: 3,900원 결제 이유를 만들어줌.

이번 달 발견한 새는 돈
27,400원

해지 후보 구독 1개
카페 예산 초과 8,000원
중복 결제 의심 1건

13-5. BudgetBucketCard

역할: 카테고리별 잔여 한도 표시.

카페
남음 8,000원

이번 주 속도가 빨라요.
다음 5일 동안 하루 1,600원씩 가능해요.

14. UX 문구 원칙

쓰면 좋은 문구

상황	문구
안전	"이번 달은 아직 안전해요."
주의	"조금만 조정하면 충분히 버틸 수 있어요."
위험	"다음 결제일 전까지 지출을 낮추는 게 좋아요."
소비 판단	"살 수는 있지만, 카드값 주간이 뽁뽁해져요."
절약 발견	"이번 달 3,900원 값은 이미 했어요."

피해야 할 문구

금지 문구	이유
"낭비했어요"	죄책감 유발
"돈 관리 못했어요"	이탈 유발
"무조건 줄이세요"	현실성 부족
"투자하세요"	서비스 목적과 다름
"가난해요"	모욕적 표현

15. 로컬 개발 폴더 구조

```
paycheck-survival-os/  
├─ app/  
│   ├─ page.tsx  
│   ├─ onboarding/  
│   │   └─ page.tsx  
│   ├─ dashboard/  
│   │   └─ page.tsx  
│   ├─ timeline/  
│   │   └─ page.tsx  
│   ├─ budget/  
│   │   └─ page.tsx  
│   ├─ subscriptions/  
│   │   └─ page.tsx  
│   ├─ decision/  
│   │   └─ page.tsx  
│   └─ settings/  
│       └─ page.tsx  
├─ components/  
│   └─ layout/  
│       └─ AppShell.tsx
```

```

├── BottomNav.tsx
├── dashboard/
│   ├── SurvivalGauge.tsx
│   ├── DailyAllowanceCard.tsx
│   ├── UpcomingBillsCard.tsx
│   └── LeakFinderCard.tsx
├── budget/
│   └── BudgetBucketCard.tsx
├── timeline/
│   └── CashflowCalendar.tsx
├── subscriptions/
│   └── SubscriptionList.tsx
├── features/
│   ├── paycycle/
│   │   ├── paycycle.types.ts
│   │   ├── paycycle.rules.ts
│   │   └── paycycle.store.ts
│   ├── budget/
│   ├── subscriptions/
│   ├── transactions/
│   └── insights/
├── lib/
│   ├── db.ts
│   ├── seed.ts
│   ├── format-money.ts
│   ├── date-utils.ts
│   └── survival-engine.ts
├── prisma/
│   └── schema.prisma
├── public/
│   ├── mock-data/
│   │   ├── transactions.csv
│   │   └── profile.json
├── styles/
│   └── globals.css

```

16. 시제품 데이터 예시

프로필

```

{
  "nickname": "민지",
  "salaryDay": 25,

```

```
"monthlyIncome": 2650000,  
"currentBalance": 1840000,  
"emergencyFloor": 300000  
}
```

고정비

```
[  
  {  
    "name": "월세",  
    "amount": 650000,  
    "dueDay": 1,  
    "category": "housing"  
  },  
  {  
    "name": "관리비",  
    "amount": 120000,  
    "dueDay": 10,  
    "category": "housing"  
  },  
  {  
    "name": "통신비",  
    "amount": 55000,  
    "dueDay": 5,  
    "category": "telecom"  
  },  
  {  
    "name": "카드값",  
    "amount": 430000,  
    "dueDay": 12,  
    "category": "card"  
  }  
]
```

구독료


```
[
  {
    "name": "유튜브 프리미엄",
    "amount": 14900,
    "billingDay": 8,
    "category": "subscription"
  },
  {
    "name": "넷플릭스",
    "amount": 17000,
    "billingDay": 15,
    "category": "subscription"
  },
  {
    "name": "쿠팡 와우",
    "amount": 7890,
    "billingDay": 3,
    "category": "subscription"
  }
]
```

17. MVP 우선순위

P0: 반드시 먼저 만들 기능

우선순위	기능	이유
1	온보딩	첫 데이터 입력 없이는 대시보드 불가
2	생존 대시보드	제품의 핵심 가치
3	하루 사용 가능 금액	매일 확인할 이유
4	고정비 타임라인	월급 생존 OS의 차별점
5	구독 레이더	3,900원 회수 명분
6	지금 사도 돼?	사용자가 재미를 느끼는 인터랙션

P1: 그다음 기능

기능	설명
CSV 거래내역 업로드	카드·계좌 데이터를 수동으로 넣는 방식
월말 회고 리포트	지난달 대비 개선점 표시
예산 자동 추천	식비·카페·쇼핑 한도 제안
결제일 알림 목업	실제 푸시 전 UI 검증

P2: 나중에 확장

기능	설명
커플·신혼 공동 관리	공동 생활비 계정
실제 금융 연동	마이데이터·카드 연동 검토
AI 코치	소비 패턴 설명
결제 기능	3,900원 구독 결제
모바일 앱	웹 검증 후 앱 전환

18. 초기 디자인 화면 목록

Figma 또는 바로 프론트 개발 기준으로 다음 화면부터 만들면 됩니다.

화면	프레임명	목적
1	01_Onboarding_Salary	월급 정보 입력
2	02_Onboarding_FixedCost	고정비 입력
3	03_Onboarding_Subscription	구독료 입력
4	04_Dashboard_Safe	안전 상태 홈
5	05_Dashboard_Warning	주의 상태 홈
6	06_Timeline_Calendar	돈 빠지는 달력
7	07_Budget_Buckets	생활비 버킷
8	08_Subscription_Radar	구독 관리
9	09_Decision_Check	지금 사도 돼?
10	10_Monthly_Review	월말 회고

19. 첫 시제품의 성공 기준

시제품은 기능이 많은 것보다 **고객이 10초 안에 “이거 필요하다”**라고 느끼는지가 중요합니다.

사용성 기준

지표	목표
온보딩 완료	3분 이내
첫 생존 점수 확인	가입 후 즉시
홈 화면 이해	10초 이내
“지금 사도 돼?” 사용	첫 이용 중 1회 이상
유료 전환 설득	“3,900원보다 더 아낄 수 있겠다” 인식

검증 질문

시제품 사용자에게 이렇게 물어보면 됩니다.

1. “이 화면을 보고 이번 달 상태를 이해했나요?”
2. “하루 사용 가능 금액이 도움이 되나요?”
3. “구독 레이터가 3,900원 결제 이유가 되나요?”
4. “기존 가게부보다 덜 귀찮나요?”
5. “월급날 전 불안이 줄어드는 느낌이 있나요?”

20. 최종 추천 방향

월급 생존 OS는 **기록형 가게부**로 만들면 경쟁이 너무 많습니다.

대신 이렇게 포지셔닝해야 합니다.

“가게부가 아니라, 월급날까지 버티는 생존 계산기.”

초기 시제품의 핵심은 딱 3개입니다.

1. **다음 월급까지 D-day**
2. **오늘 하루 사용 가능 금액**
3. **다음 7일 빠질 돈**

이 3개가 한 화면에서 강하게 보이면, 24~39세 사회초년생에게 월 3,900원짜리 서비스로 설득력이 생깁니다.

